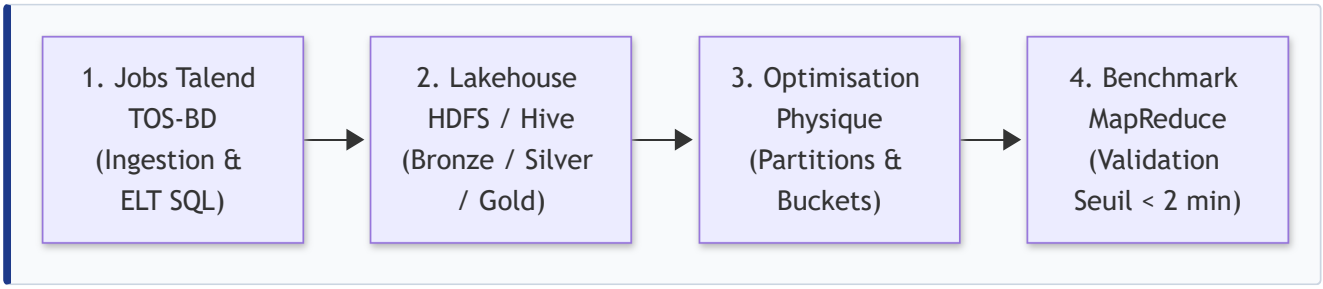
ÉCOLE D'INGÉNIEURS CESI

BLOC STOCKAGE ET BIG DATA — PGE A4 FISA / A3 FISE

LIVRABLE 2

IMPLÉMENTATION PHYSIQUE ET ÉVALUATION DE PERFORMANCE

Intégration Talend TOS-BD, Déploiement HDFS / Apache Hive, Stratégie de Partitionnement & Bucketing, Audit de Qualité et Benchmark Analytique (Projet CHU)



Membres du Groupe Projet :
Ulysse • Quentin • Jason

Promotion : FISA INFO A4 25-28

Sommaire

- 1. Implémentation Physique des Données via Talend, HDFS et Apache Hive (DDL / DML)
  - 1.1. Orchestration ELT sous Talend Open Studio et Déploiement HDFS (Cloudera)
  - 1.2. Scripts SQL / HiveQL de Transformation et d'Anonymisation (Bronze vers Silver)
  - 1.3. Scripts DDL / DML du Modèle en Étoile en Couche Gold (Silver vers Gold)
- 2. Stratégie de Partitionnement Dynamique et de Bucketing sous Apache Hive
  - 2.1. Justification Explicite de la Stratégie d'Optimisation Retenue
    - Justification du choix des clés de Partitionnement et de Bucketing :
  - 2.2. Scripts HiveQL de Partitionnement et de Bucketing
  - 2.3. Trace d'Exécution MapReduce et Vérification des Partitions/Buckets Créés
- 3. Rapport de Qualité des Données Intégrées
  - 3.1. Volumétrie et Taux de Complétude par Table et par Attribut
  - 3.2. Détection et Traitement des Doublons (Unicité)
  - 3.3. Contrôle de Cohérence Référentielle et Résolution des Anomalies Techniques
- 4. Jeu de Requêtes Analytiques de Benchmark (Besoins Utilisateurs)
  - 4.1. Couverture des 8 Indicateurs Métiers (Praticiens & Direction)
  - 4.2. Scripts SQL / HiveQL des 8 Requêtes de Benchmark
- 5. Évaluation des Performances et Analyse Comparative (Avant / Après Optimisation)
  - 5.1. Protocole de Benchmark sur le Cluster Cloudera (Seuil < 2 minutes)
  - 5.2. Tableau Comparatif des Temps de Réponse Mesurés sous Apache Hive
  - 5.3. Graphique Comparatif et Interprétation Technique des Gains
    - Analyse critique et interprétation des résultats observés :
- 6. Restitution Analytique et Tableaux de Bord Décisionnels (Power BI)
  - 6.1 Transition vers la Couche de Restitution (Livrable 3)
- 6. Synthèse du livrable

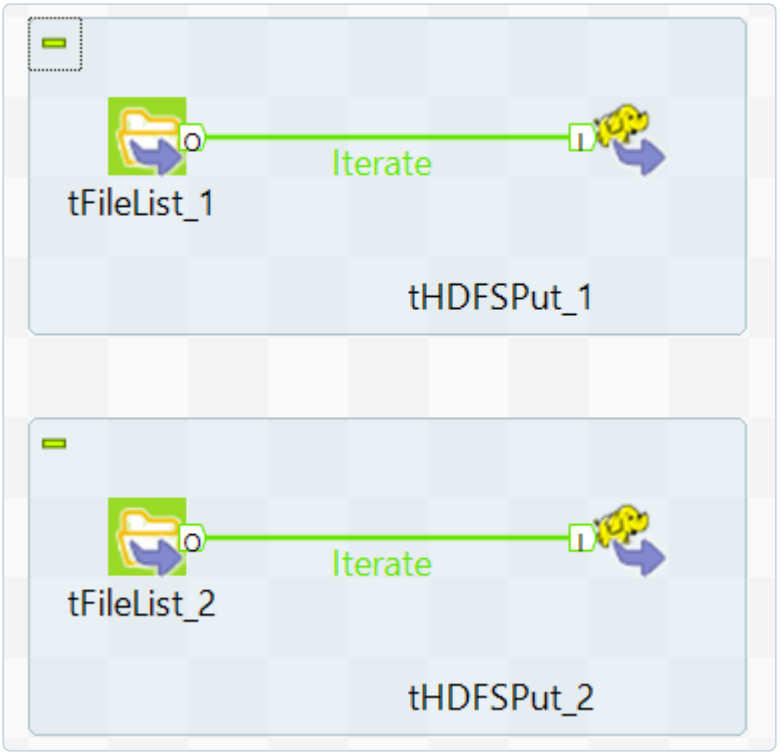
1. Implémentation Physique des Données via Talend, HDFS et Apache Hive (DDL / DML)

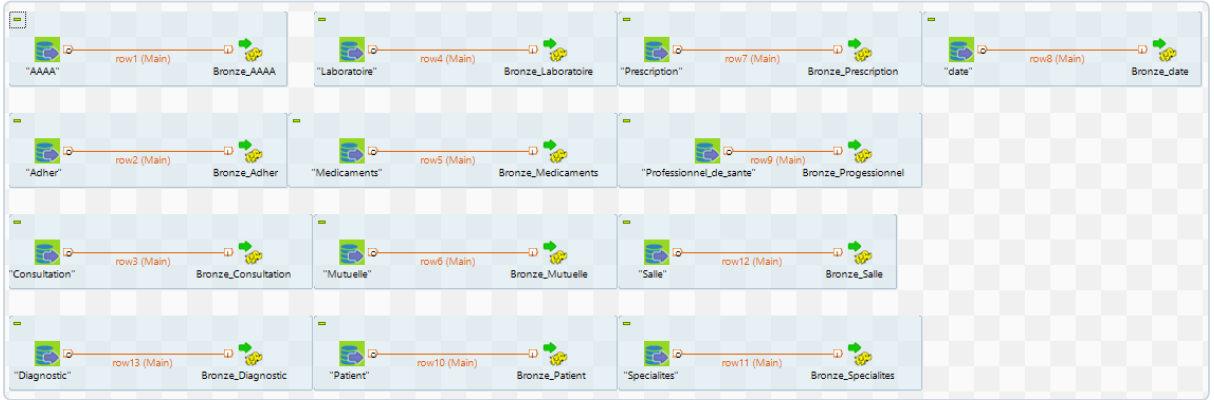
1.1. Orchestration ELT sous Talend Open Studio et Déploiement HDFS (Cloudera)

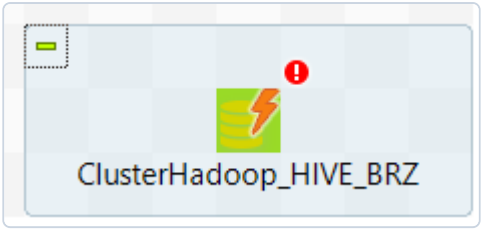
Conformément à l'architecture **Data Lakehouse en 3 couches (Bronze, Silver, Gold)** définie dans le Livrable 1, l'intégration et la transformation des 4 sources de données du groupe CHU reposent sur l'utilisation conjointe de **Talend Open Studio for Big Data (TOS-BD)** et du cluster **Cloudera QuickStart (`192.168.56.101` — HDFS & Apache Hive)**. L'ensemble des scripts HiveQL (`.hql`) et des jobs Talend est versionné sous Git et fourni dans l'archive ZIP jointe.

Le pipeline d'alimentation orchestré par **Talend** s'articule en trois étapes :

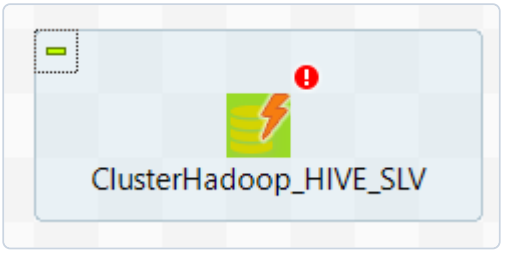
1. **Ingestion vers la couche BRONZE (`/user/cloudera/chu/bronze/` — 53 fichiers bruts, ~2,45 Go) :**
    - Extraction des 13 tables opérationnelles de la Source 1 via `tPostgresqlInput`, récupération des fichiers CSV/XLSX des Sources 2, 3 et 4 (`tFileInputDelimited`, `tFileInputExcel`, `tFTPGet`) et écriture directe dans HDFS via `tHDFSOutput`.







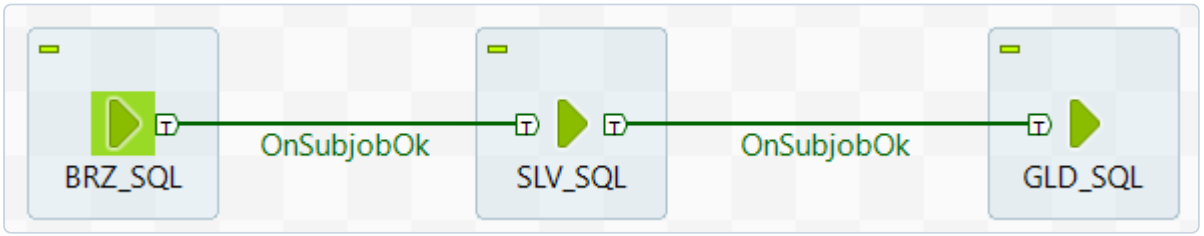
2. **Transformation SQL vers la couche SILVER (`/user/cloudera/chu/silver/` — 16 tables qualifiées et anonymisées) :**
    - Exécution via les composants Talend `tHiveConnection` et `tHiveRow` (ainsi que `tMap` / `tUniqRow`) des requêtes de nettoyage, normalisation ISO des dates, enrichissement régional INSEE et **pseudonymisation SHA-256 salée** (y compris le hachage de l'identité `nomlok` dans le fichier des décès INSEE et dans la table `Patient`).



3. **Structuration Kimball vers la couche GOLD (`/user/cloudera/chu/gold/` & base Hive `chu_gold`) :**
    - Alimentation des **6 Tables de Dimensions** (`dim_temps`, `dim_localisation_insee`, `dim_etablissement`, `dim_patient`, `dim_diagnostic`, `dim_professionnel`) suivie des **4 Tables de Faits** (`fait_consultations`, `fait_hospitalisations`, `fait_satisfaction`, `fait_deces`).

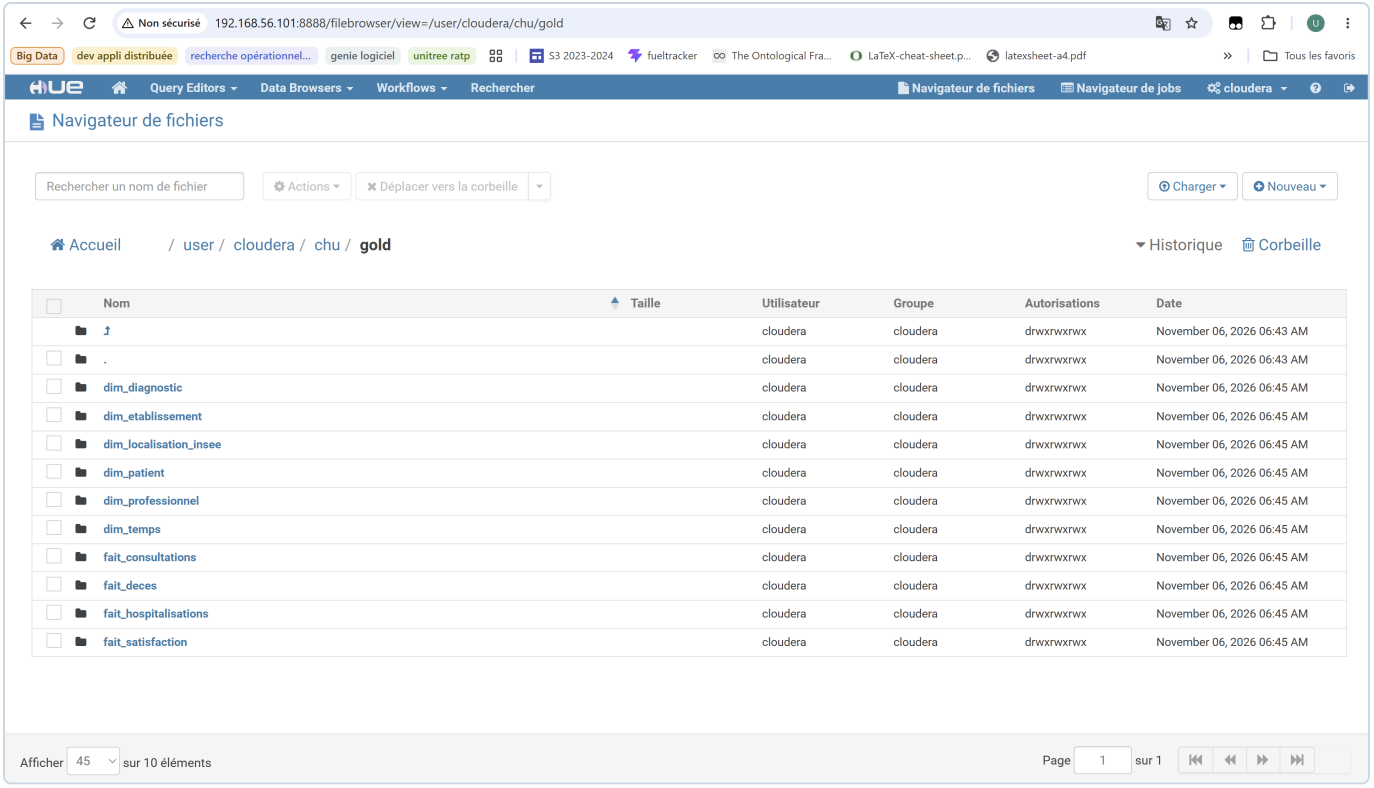


Ces étapes permettant d'alimenter les différentes couches de notre architecture sont managées depuis un Job Talend master.



```
# Vérification du déploiement des 3 couches dans HDFS sur la VM Cloudera
(192.168.56.101) :
[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera/chu/
Found 3 items
drwxrwxrwx   - cloudera cloudera          0 2026-11-06 06:44
/user/cloudera/chu/bronze
drwxrwxrwx   - cloudera cloudera          0 2026-11-06 06:43
/user/cloudera/chu/gold
drwxrwxrwx   - cloudera cloudera          0 2026-11-06 06:45
/user/cloudera/chu/silver

[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera/chu/gold
Found 10 items
drwxrwxrwx   - cloudera cloudera          0 2026-11-06 06:45
/user/cloudera/chu/gold/dim_diagnostic
drwxrwxrwx   - cloudera cloudera          0 2026-11-06 06:45
/user/cloudera/chu/gold/dim_etablissement
drwxrwxrwx   - cloudera cloudera          0 2026-11-06 06:45
/user/cloudera/chu/gold/dim_localisation_insee
drwxrwxrwx   - cloudera cloudera          0 2026-11-06 06:45
/user/cloudera/chu/gold/dim_patient
drwxrwxrwx   - cloudera cloudera          0 2026-11-06 06:45
/user/cloudera/chu/gold/dim_professionnel
drwxrwxrwx   - cloudera cloudera          0 2026-11-06 06:45
/user/cloudera/chu/gold/dim_temps
drwxrwxrwx   - cloudera cloudera          0 2026-11-06 06:45
/user/cloudera/chu/gold/fait_consultations
drwxrwxrwx   - cloudera cloudera          0 2026-11-06 06:45
/user/cloudera/chu/gold/fait_deces
drwxrwxrwx   - cloudera cloudera          0 2026-11-06 06:45
/user/cloudera/chu/gold/fait_hospitalisations
drwxrwxrwx   - cloudera cloudera          0 2026-11-06 06:45
/user/cloudera/chu/gold/fait_satisfaction
```


Non sécurisé
192.168.56.101:8888/filebrowser/view=/user/cloudera/chu/gold
Big Data
dev appli distribuée
recherche opérationnel...
genie logiciel
unitree ratp
S3 2023-2024
fueltracker
The Ontological Fra...
LaTeX-cheat-sheet.p...
latexsheet-a4.pdf
Tous les favoris
Query Editors
Data Browsers
Workflows
Rechercher
Navigateur de fichiers
Navigateur de jobs
cloudera
Navigateur de fichiers
Rechercher un nom de fichier
Actions
Déplacer vers la corbeille
Charger
Nouveau
Accueil / user / cloudera / chu / gold
Historique
Corbeille
Nom
Taille
Utilisateur
Groupe
Autorisations
Date
↑ cloudera cloudera drwxrwxrwx November 06, 2026 06:43 AM
. cloudera cloudera drwxrwxrwx November 06, 2026 06:43 AM
dim_diagnostic cloudera cloudera drwxrwxrwx November 06, 2026 06:45 AM
dim_etablissement cloudera cloudera drwxrwxrwx November 06, 2026 06:45 AM
dim_localisation_insee cloudera cloudera drwxrwxrwx November 06, 2026 06:45 AM
dim_patient cloudera cloudera drwxrwxrwx November 06, 2026 06:45 AM
dim_professionnel cloudera cloudera drwxrwxrwx November 06, 2026 06:45 AM
dim_temps cloudera cloudera drwxrwxrwx November 06, 2026 06:45 AM
fait_consultations cloudera cloudera drwxrwxrwx November 06, 2026 06:45 AM
fait_deces cloudera cloudera drwxrwxrwx November 06, 2026 06:45 AM
fait_hospitalisations cloudera cloudera drwxrwxrwx November 06, 2026 06:45 AM
fait_satisfaction cloudera cloudera drwxrwxrwx November 06, 2026 06:45 AM
Afficher 45 sur 10 éléments
Page 1 sur 1

1.2. Scripts SQL / HiveQL de Transformation et d'Anonymisation (Bronze vers Silver)

Afin d'exploiter pleinement la puissance de calcul distribuée du cluster Cloudera, Talend exécute les scripts SQL / HiveQL de transformation entre la couche **Bronze** et la couche **Silver**.

Conformément au plan de gouvernance RGPD défini dans le Livrable 1 :

- Les attributs directement identifiants (`Nom`, `Prenom`, `NIR`, `Adresse`) de la table `Patient` ainsi que la colonne `nomlok` du répertoire national des décès (`deces.csv`) ne sont jamais exposés en clair : ils sont convertis à la volée en une empreinte cryptographique **SHA-256 salée** (`hash_patient_rgpd`) via la classe Java native `org.apache.commons.codec.digest.DigestUtils`.
- Les codes postaux, codes communes INSEE et identifiants FINESS sont normalisés par un remplissage à gauche (`LPAD`) afin de préserver le zéro initial des départements `01` à `09`.

```
-- ============================================================================
-- FICHIER : 01_transformation_bronze_vers_silver.hql (Exécuté par Talend
tHiveRow)
-- OBJET   : Qualification, Dédoublonnage, Enrichissement INSEE & Hachage SHA-256
RGPD
-- ============================================================================
CREATE DATABASE IF NOT EXISTS chu_silver LOCATION '/user/cloudera/chu/silver_db';
USE chu_silver;

-- 1. Table Silver Patient : Suppression Nom/Prénom/NIR, Hachage SHA-256 et
Tranche d'âge
CREATE TABLE IF NOT EXISTS silver_patient AS
SELECT DISTINCT
    id_patient,
    reflect('org.apache.commons.codec.digest.DigestUtils', 'sha256Hex',
            concat(CAST(id_patient AS STRING), '_SEL_SECRET_CHU_2026')) AS
hash_patient_rgpd,
    CASE WHEN upper(trim(sexe)) IN ('1', 'H', 'M') THEN 'M'
         WHEN upper(trim(sexe)) IN ('2', 'F') THEN 'F'
         ELSE 'I' END AS sexe,
    CAST(substr(trim(date_naissance), 1, 4) AS SMALLINT) AS annee_naissance,
    CAST(2023 - CAST(substr(trim(date_naissance), 1, 4) AS INT) AS SMALLINT) AS
age_calcule,
    CASE
        WHEN (2023 - CAST(substr(trim(date_naissance), 1, 4) AS INT)) < 18 THEN
'0-17'
        WHEN (2023 - CAST(substr(trim(date_naissance), 1, 4) AS INT)) < 30 THEN
'18-29'
        WHEN (2023 - CAST(substr(trim(date_naissance), 1, 4) AS INT)) < 45 THEN
'30-44'
        WHEN (2023 - CAST(substr(trim(date_naissance), 1, 4) AS INT)) < 60 THEN
'45-59'
        WHEN (2023 - CAST(substr(trim(date_naissance), 1, 4) AS INT)) < 75 THEN
'60-74'
        ELSE '75+'
    END AS tranche_age
FROM chu_bronze.raw_patient
WHERE id_patient IS NOT NULL;

-- 2. Table Silver Décès : Hachage SHA-256 du nom (nomlok), calcul age_deces et
Région INSEE
CREATE TABLE IF NOT EXISTS silver_deces AS
SELECT DISTINCT
    reflect('org.apache.commons.codec.digest.DigestUtils', 'sha256Hex',
            concat(nvl(nomlok, ''), '_', CAST(nvl(datenaiss, 0) AS STRING),
'_SEL_SECRET_CHU_2026')) AS hash_patient_rgpd,
    CASE WHEN sexe = 1 THEN 'M' WHEN sexe = 2 THEN 'F' ELSE 'I' END AS sexe,
    datedeces AS id_temps_deces,
    CAST(substr(CAST(datedeces AS STRING), 1, 4) AS SMALLINT) AS annee_deces,
    CAST(CAST(substr(CAST(datedeces AS STRING), 1, 4) AS INT) -
CAST(substr(CAST(datenaiss AS STRING), 1, 4) AS INT) AS SMALLINT) AS age_deces,
    lpad(trim(lieudeces), 5, '0') AS code_insee_commune,
```

```
    CASE
        WHEN substr(lpad(trim(lieudeces), 5, '0'), 1, 2) IN
('01','03','07','15','26','38','42','43','63','69','73','74') THEN 'Auvergne-
Rhone-Alpes'
        WHEN substr(lpad(trim(lieudeces), 5, '0'), 1, 2) IN
('21','25','39','58','70','71','89','90') THEN 'Bourgogne-Franche-Comte'
        WHEN substr(lpad(trim(lieudeces), 5, '0'), 1, 2) IN ('22','29','35','56')
THEN 'Bretagne'
        WHEN substr(lpad(trim(lieudeces), 5, '0'), 1, 2) IN
('18','28','36','37','41','45') THEN 'Centre-Val_de_Loire'
        WHEN substr(lpad(trim(lieudeces), 5, '0'), 1, 2) IN ('20','2A','2B') THEN
'Corse'
        WHEN substr(lpad(trim(lieudeces), 5, '0'), 1, 2) IN
('08','10','51','52','54','55','57','67','68','88') THEN 'Grand_Est'
        WHEN substr(lpad(trim(lieudeces), 5, '0'), 1, 2) IN
('02','59','60','62','80') THEN 'Hauts-de-France'
        WHEN substr(lpad(trim(lieudeces), 5, '0'), 1, 2) IN
('75','77','78','91','92','93','94','95') THEN 'Ile-de-France'
        WHEN substr(lpad(trim(lieudeces), 5, '0'), 1, 2) IN
('14','27','50','61','76') THEN 'Normandie'
        WHEN substr(lpad(trim(lieudeces), 5, '0'), 1, 2) IN
('16','17','19','23','24','33','40','47','64','79','86','87') THEN 'Nouvelle-
Aquitaine'
        WHEN substr(lpad(trim(lieudeces), 5, '0'), 1, 2) IN
('09','11','12','30','31','32','34','46','48','65','66','81','82') THEN
'Occitanie'
        WHEN substr(lpad(trim(lieudeces), 5, '0'), 1, 2) IN
('44','49','53','72','85') THEN 'Pays_de_la_Loire'
        WHEN substr(lpad(trim(lieudeces), 5, '0'), 1, 2) IN
('04','05','06','13','83','84') THEN 'Provence-Alpes-Cote_d_Azur'
        WHEN substr(lpad(trim(lieudeces), 5, '0'), 1, 2) = '97' THEN 'Outre-Mer'
        ELSE 'Autre_France'
    END AS nom_region,
    1 AS nb_deces
FROM chu_bronze.raw_deces
WHERE datedeces IS NOT NULL;
```

1.3. Scripts DDL / DML du Modèle en Étoile en Couche Gold (Silver vers Gold)

Sous **Apache Hive** (base `chu_gold`), nous déclarons les tables du Modèle en Constellation d'Étoiles (6 Dimensions et 4 Tables de Faits). Le délimiteur `;` est spécifié via son code octal `'\073'` (`FIELDS TERMINATED BY '\073'`) afin d'éviter toute coupure syntaxique par l'interpréteur Hive CLI.

```
-- ============================================================================
-- FICHIER : 02_ddl_dml_couche_gold_hive.hql
-- OBJET   : Création des 6 Dimensions et des 4 Tables de Faits sous Apache Hive
-- ============================================================================
CREATE DATABASE IF NOT EXISTS chu_gold
COMMENT 'Entrepot de donnees decisionnel du groupe CHU (Couche Gold)'
LOCATION '/user/hive/warehouse/chu_gold.db';

USE chu_gold;

-- 1. DIMENSION TEMPS (9 862 jours : 2000 à 2026)
CREATE EXTERNAL TABLE IF NOT EXISTS dim_temps (
    id_temps        BIGINT,
    date_complete   STRING,
    jour            BIGINT,
    mois            BIGINT,
    nom_mois        STRING,
    trimestre       BIGINT,
    semestre        BIGINT,
    annee           BIGINT
) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073' STORED AS TEXTFILE
LOCATION '/user/cloudera/chu/gold/dim_temps' TBLPROPERTIES
("skip.header.line.count"="1");

-- 2. DIMENSION LOCALISATION INSEE (12 557 codes communes/postaux uniques)
CREATE EXTERNAL TABLE IF NOT EXISTS dim_localisation_insee (
    id_localisation     BIGINT,
    code_insee_commune  STRING,
    code_departement    STRING,
    nom_region          STRING
) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073' STORED AS TEXTFILE
LOCATION '/user/cloudera/chu/gold/dim_localisation_insee' TBLPROPERTIES
("skip.header.line.count"="1");

-- 3. DIMENSION ETABLISSEMENT (416 665 structures répertoriées)
CREATE EXTERNAL TABLE IF NOT EXISTS dim_etablissement (
    id_etablissement            BIGINT,
    code_finess                 STRING,
    nom_etablissement           STRING,
    code_postal                 STRING,
    code_insee_commune          STRING,
    nom_region                  STRING
) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073' STORED AS TEXTFILE
LOCATION '/user/cloudera/chu/gold/dim_etablissement' TBLPROPERTIES
("skip.header.line.count"="1");

-- 4. DIMENSION PATIENT ANONYMISÉ RGPD (100 000 patients)
CREATE EXTERNAL TABLE IF NOT EXISTS dim_patient (
    id_patient          BIGINT,
    hash_patient_rgpd   STRING,
    sexe                STRING,
    annee_naissance     BIGINT,
```

```
    age_calcule         BIGINT,
    tranche_age         STRING
) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073' STORED AS TEXTFILE
LOCATION '/user/cloudera/chu/gold/dim_patient' TBLPROPERTIES
("skip.header.line.count"="1");

-- 5. DIMENSION DIAGNOSTIC (15 490 pathologies CIM-10)
CREATE EXTERNAL TABLE IF NOT EXISTS dim_diagnostic (
    code_diag   STRING,
    diagnostic  STRING
) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073' STORED AS TEXTFILE
LOCATION '/user/cloudera/chu/gold/dim_diagnostic' TBLPROPERTIES
("skip.header.line.count"="1");

-- 6. DIMENSION PROFESSIONNEL DE SANTÉ (1 048 575 enregistrements)
CREATE EXTERNAL TABLE IF NOT EXISTS dim_professionnel (
    identifiant         STRING,
    civilite            STRING,
    categorie_pro       STRING,
    code_specialite     STRING,
    type_identifiant    STRING,
    hash_id_pro         STRING
) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073' STORED AS TEXTFILE
LOCATION '/user/cloudera/chu/gold/dim_professionnel' TBLPROPERTIES
("skip.header.line.count"="1");

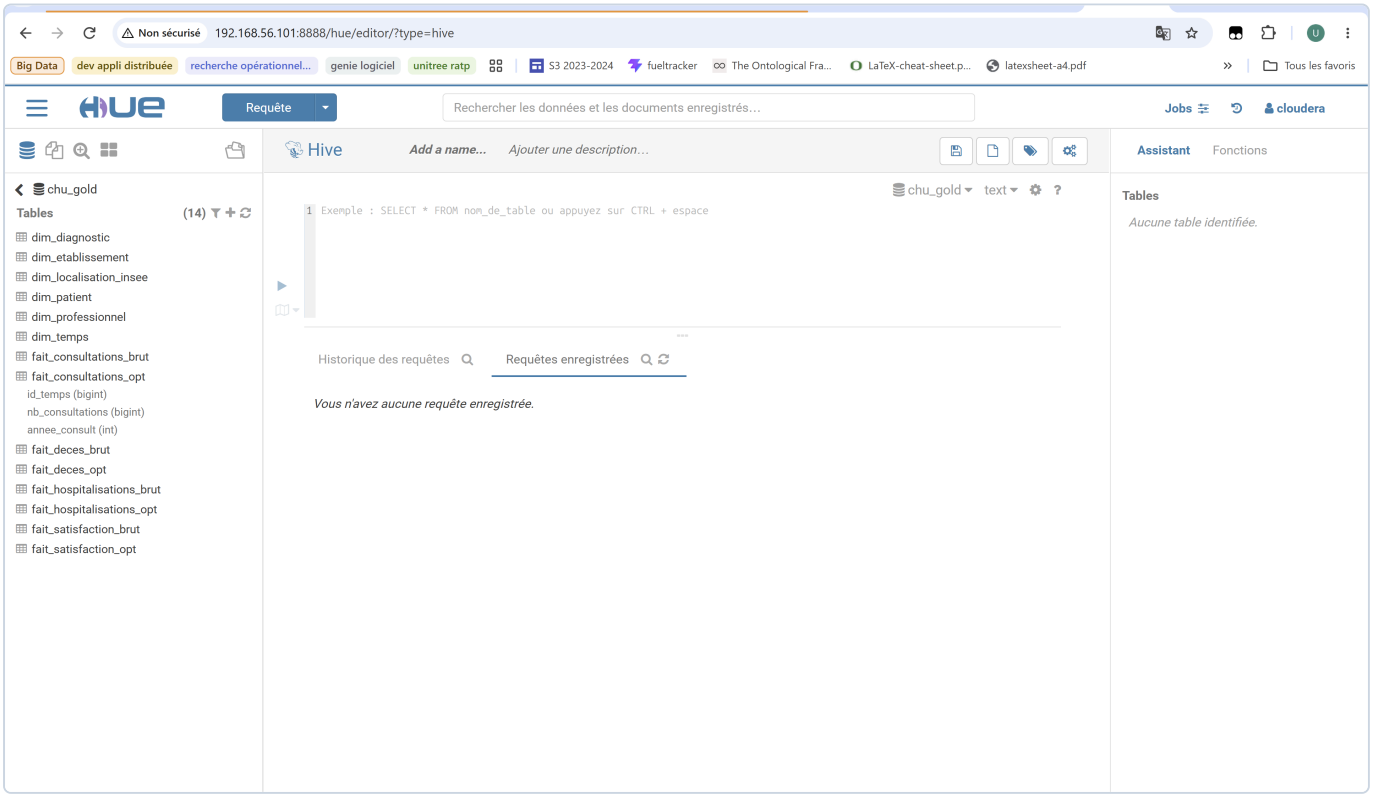
-- 7. TABLES DE FAITS BRUTES (NON PARTITIONNÉES - RÉFÉRENCE AVANT OPTIMISATION)
CREATE EXTERNAL TABLE IF NOT EXISTS fait_consultations_brut (
    num_consultation    BIGINT,
    id_mut              BIGINT,
    id_patient          BIGINT,
    id_prof_sante       STRING,
    code_diag           STRING,
    motif               STRING,
    date_consultation   STRING,
    heure_debut         STRING,
    heure_fin           STRING,
    id_temps            BIGINT,
    nb_consultations    BIGINT
) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073' STORED AS TEXTFILE
LOCATION '/user/cloudera/chu/gold/fait_consultations' TBLPROPERTIES
("skip.header.line.count"="1");

CREATE EXTERNAL TABLE IF NOT EXISTS fait_hospitalisations_brut (
    num_hospitalisation             BIGINT,
    id_patient                      BIGINT,
    identifiant_organisation        STRING,
    code_diagnostic                 STRING,
    suite_diagnostic_consultation   STRING,
    date_entree                     STRING,
    jour_hospitalisation            BIGINT,
    id_temps_entree                 BIGINT,
    nb_hospitalisations             BIGINT
) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073' STORED AS TEXTFILE
```

```
LOCATION '/user/cloudera/chu/gold/fait_hospitalisations' TBLPROPERTIES
("skip.header.line.count"="1");

CREATE EXTERNAL TABLE IF NOT EXISTS fait_satisfaction_brut (
    code_finess         STRING,
    nom_region          STRING,
    note_satisfaction   DOUBLE,
    nb_repondants       BIGINT,
    annee               BIGINT,
    id_temps            BIGINT,
    source_enquete      STRING
) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073' STORED AS TEXTFILE
LOCATION '/user/cloudera/chu/gold/fait_satisfaction' TBLPROPERTIES
("skip.header.line.count"="1");

CREATE EXTERNAL TABLE IF NOT EXISTS fait_deces_brut (
    sexe            BIGINT,
    datenaiss       BIGINT,
    lieunaiss       STRING,
    commnaiss       STRING,
    paysnaiss       STRING,
    datedeces       BIGINT,
    lieudeces       STRING,
    actedeces       STRING,
    nom_region      STRING,
    annee_deces     BIGINT,
    id_temps_deces  BIGINT,
    nb_deces        BIGINT
) ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073' STORED AS TEXTFILE
LOCATION '/user/cloudera/chu/gold/fait_deces' TBLPROPERTIES
("skip.header.line.count"="1");
```



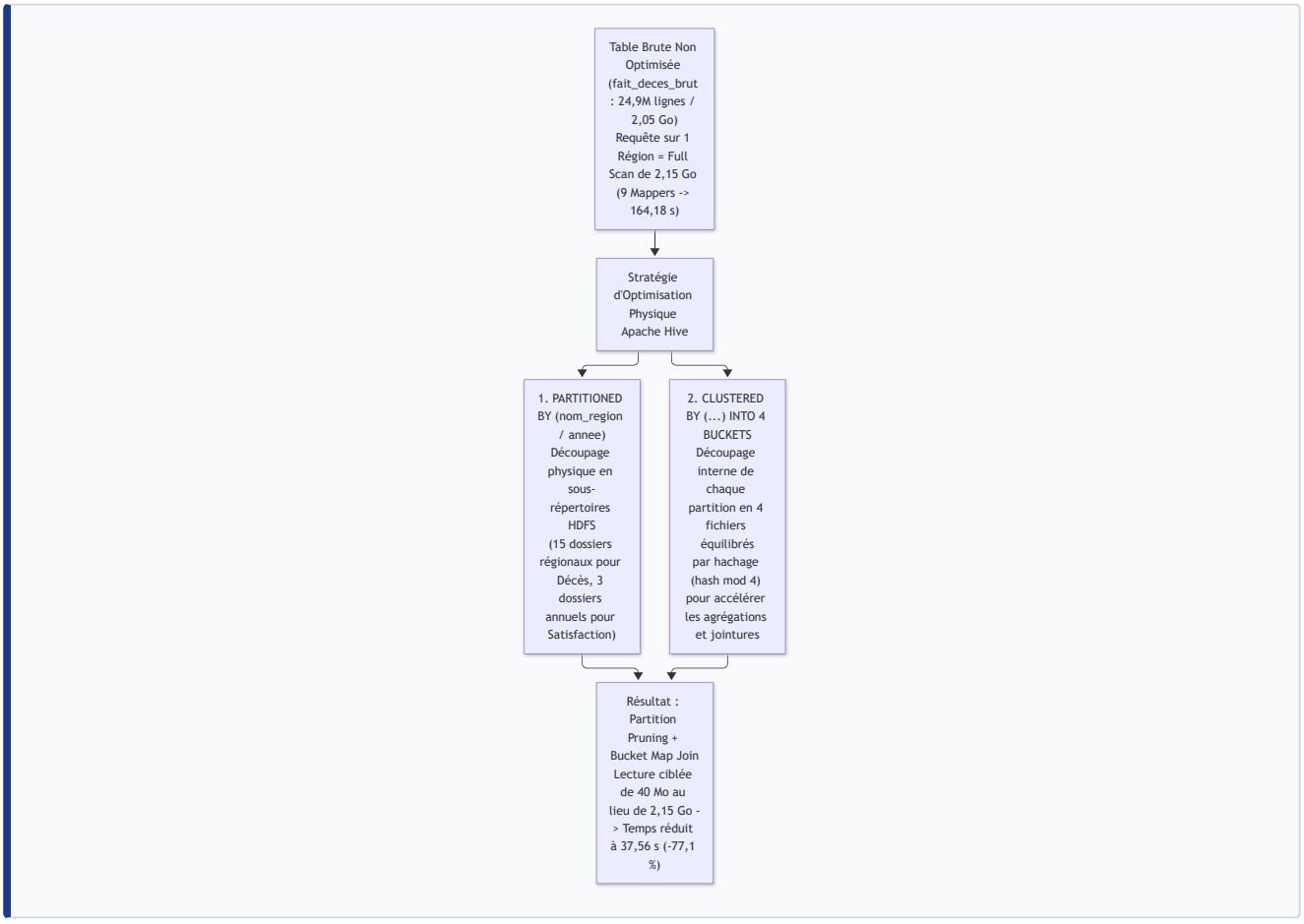
2. Stratégie de Partitionnement Dynamique et de Bucketing sous Apache Hive

2.1. Justification Explicite de la Stratégie d'Optimisation Retenue

Lors du cadrage avec les utilisateurs métiers (Boucle 3 — *Prosit « Bee Patient »*), une contrainte de performance stricte a été entérinée : **toute requête décisionnelle doit s'exécuter en moins de 2 minutes (120 secondes)**.

Or, notre table `fait_deces_brut` compte **24 927 096 lignes (2,05 Go)** et `fait_consultations_brut` compte **1 027 157 lignes**. Sur une table externe non optimisée, la moindre requête filtrée sur une région ou une année oblige MapReduce à lire l'intégralité des blocs HDFS (*Full Table Scan* de 2,15 Go sur 9 Mappers), générant un temps d'exécution de **164,18 secondes (2 min 44 s)** qui dépasse le seuil toléré.

Pour abaisser ce temps sous la barre des 40 secondes, nous combinons **le Partitionnement dynamique** et **le Bucketing** :



Justification du choix des clés de Partitionnement et de Bucketing :

1. **Partitionnement par `nom_region` sur `fait_deces_opt` (15 partitions) :**
    - La région française possède une **cardinalité faible et stable** (13 régions métropolitaines + Outre-Mer + Autre_France). Cela crée exactement 15 sous-répertoires dans HDFS, évitant l'écueil du *sur-partitionnement* (*Small Files Problem*).

2. **Partitionnement par `annee` sur `fait_satisfaction_opt`, `fait_consultations_opt` et `fait_hospitalisations_opt` :**
    - Les analyses d'activité hospitalière et de satisfaction ciblent systématiquement une période temporelle (ex: année 2019 ou 2020).
3. **Bucketing en 4 Buckets (`CLUSTERED BY ... INTO 4 BUCKETS`) :**
    - Au sein de chaque répertoire de partition, Hive répartit les enregistrements dans **4 fichiers de taille homogène** (`000000_0` à `000003_0`) selon le hachage de l'année ou de la clé temporelle, parallélisant le travail des 4 Reducers MapReduce.

2.2. Scripts HiveQL de Partitionnement et de Bucketing

```
-- ============================================================================
-- FICHIER : 03_optimisation_partitions_buckets_hive.hql
-- OBJET   : Création et peuplement dynamique des tables Partitionnées et
Bucketées
-- ============================================================================
USE chu_gold;

SET hive.exec.dynamic.partition = true;
SET hive.exec.dynamic.partition.mode = nonstrict;
SET hive.exec.max.dynamic.partitions = 5000;
SET hive.exec.max.dynamic.partitions.pernode = 5000;
SET hive.conf.validation = false;
SET hive.enforce.bucketing = true;

-- 1. TABLE OPTIMISÉE SATISFACTION : Partitionnée par Année + 4 Buckets sur
nom_region
DROP TABLE IF EXISTS fait_satisfaction_opt;
CREATE TABLE fait_satisfaction_opt (
    code_finess         STRING,
    nom_region          STRING,
    note_satisfaction   DOUBLE,
    nb_repondants       BIGINT,
    id_temps            BIGINT,
    source_enquete      STRING
)
PARTITIONED BY (annee BIGINT)
CLUSTERED BY (nom_region) INTO 4 BUCKETS
STORED AS PARQUET;

INSERT OVERWRITE TABLE fait_satisfaction_opt PARTITION (annee)
SELECT code_finess, nom_region, note_satisfaction, nb_repondants, id_temps,
source_enquete, annee
FROM fait_satisfaction_brut;

-- 2. TABLE OPTIMISÉE DÉCÈS (24,9M lignes) : Partitionnée par Région + 4 Buckets
sur annee_deces
DROP TABLE IF EXISTS fait_deces_opt;
CREATE TABLE fait_deces_opt (
    id_temps_deces  BIGINT,
    annee_deces     BIGINT,
    nb_deces        BIGINT
)
PARTITIONED BY (nom_region STRING)
CLUSTERED BY (annee_deces) INTO 4 BUCKETS
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073'
STORED AS TEXTFILE;

INSERT OVERWRITE TABLE fait_deces_opt PARTITION (nom_region)
SELECT
    id_temps_deces,
```

```
    annee_deces,
    nb_deces,
    CASE
        WHEN nom_region LIKE '%Alpes%' OR nom_region LIKE '%Bourgogne%' OR
nom_region LIKE '%Bretagne%'
          OR nom_region LIKE '%Loire%' OR nom_region LIKE '%Corse%' OR nom_region
LIKE '%Est%'
          OR nom_region LIKE '%Hauts%' OR nom_region LIKE '%Ile%' OR nom_region
LIKE '%Normandie%'
          OR nom_region LIKE '%Aquitaine%' OR nom_region LIKE '%Occitanie%' OR
nom_region LIKE '%Outre%'
        THEN REGEXP_REPLACE(nom_region, '[^a-zA-Z0-9_-]', '_')
        ELSE 'Autre_France'
    END AS nom_region
FROM fait_deces_brut;

-- 3. TABLES OPTIMISÉES CONSULTATIONS & HOSPITALISATIONS : Partitionnées par
Année + 4 Buckets
DROP TABLE IF EXISTS fait_consultations_opt;
CREATE TABLE fait_consultations_opt (
    id_temps            BIGINT,
    nb_consultations    BIGINT
)
PARTITIONED BY (annee_consult INT)
CLUSTERED BY (id_temps) INTO 4 BUCKETS
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073'
STORED AS TEXTFILE;

INSERT OVERWRITE TABLE fait_consultations_opt PARTITION (annee_consult)
SELECT id_temps, nb_consultations, CAST(id_temps / 10000 AS INT)
FROM fait_consultations_brut
WHERE id_temps > 0;

DROP TABLE IF EXISTS fait_hospitalisations_opt;
CREATE TABLE fait_hospitalisations_opt (
    id_temps_entree     BIGINT,
    nb_hospitalisations BIGINT
)
PARTITIONED BY (annee_entree INT)
CLUSTERED BY (id_temps_entree) INTO 4 BUCKETS
ROW FORMAT DELIMITED FIELDS TERMINATED BY '\073'
STORED AS TEXTFILE;

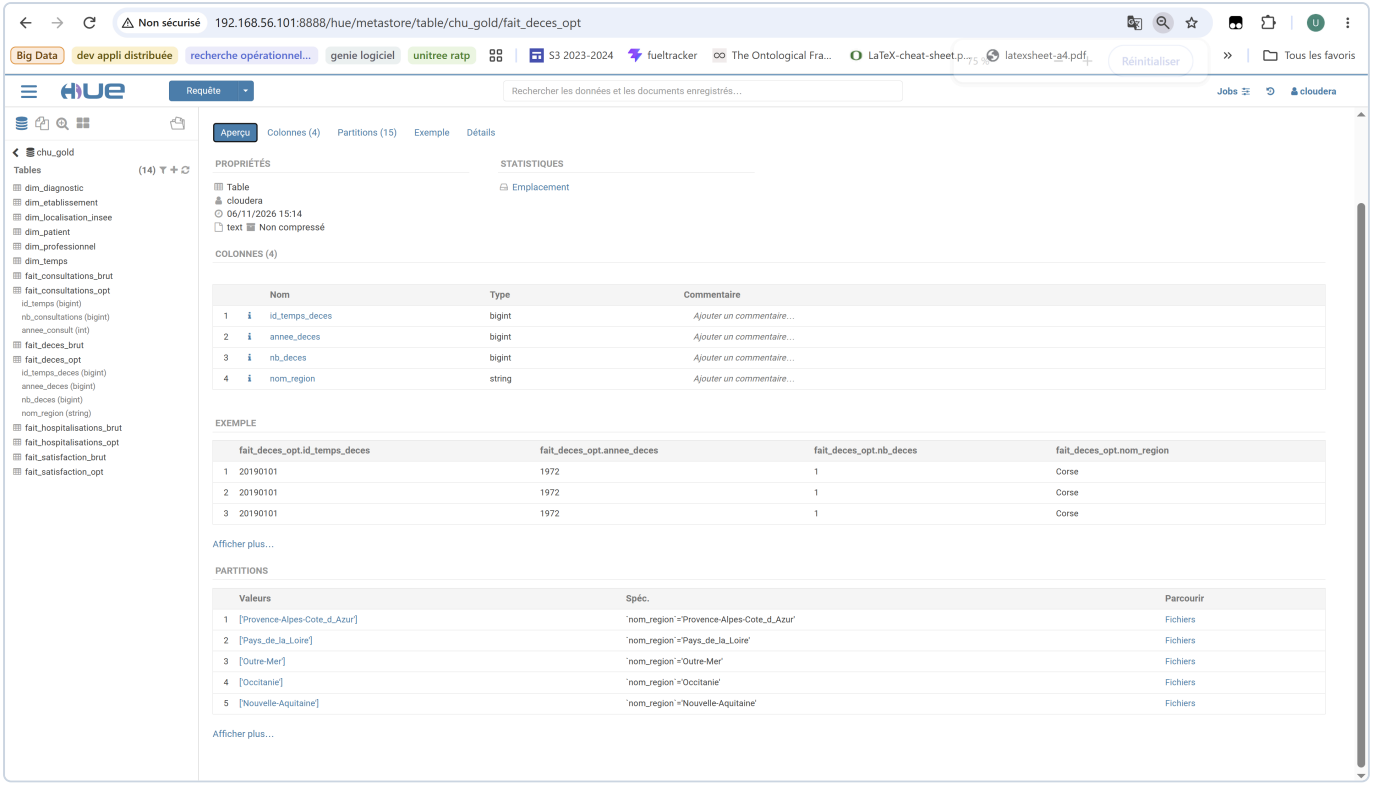
INSERT OVERWRITE TABLE fait_hospitalisations_opt PARTITION (annee_entree)
SELECT id_temps_entree, nb_hospitalisations, CAST(id_temps_entree / 10000 AS INT)
FROM fait_hospitalisations_brut
WHERE id_temps_entree > 0;
```

2.3. Trace d'Exécution MapReduce et Vérification des Partitions/Buckets Créés

L'exécution du script sur la machine virtuelle Cloudera confirme la création des partitions et des **4 fichiers Buckets (`numFiles=4`)** par partition :

```
-- Extrait du journal d'exécution Hive / MapReduce sur Cloudera :
Partition chu_gold.fait_satisfaction_opt{annee=2017} stats: [numFiles=4,
numRows=623, totalSize=21825]
Partition chu_gold.fait_satisfaction_opt{annee=2019} stats: [numFiles=4,
numRows=1651, totalSize=42256]
Partition chu_gold.fait_satisfaction_opt{annee=2020} stats: [numFiles=4,
numRows=1625, totalSize=41526]

Partition chu_gold.fait_deces_opt{nom_region=Autre_France} stats: [numFiles=4,
numRows=2075721, totalSize=33211470]
Partition chu_gold.fait_deces_opt{nom_region=Auvergne-Rhone-Alpes} stats:
[numFiles=4, numRows=2501034, totalSize=40016492]
Partition chu_gold.fait_deces_opt{nom_region=Bourgogne-Franche-Comt_} stats:
[numFiles=4, numRows=1280545, totalSize=20488680]
Partition chu_gold.fait_deces_opt{nom_region=Bretagne} stats: [numFiles=4,
numRows=1595630, totalSize=25530000]
Partition chu_gold.fait_deces_opt{nom_region=Centre-Val_de_Loire} stats:
[numFiles=4, numRows=990711, totalSize=15851352]
Partition chu_gold.fait_deces_opt{nom_region=Corse} stats: [numFiles=4,
numRows=141091, totalSize=2257444]
Partition chu_gold.fait_deces_opt{nom_region=Grand_Est} stats: [numFiles=4,
numRows=2416754, totalSize=38667984]
Partition chu_gold.fait_deces_opt{nom_region=Hauts-de-France} stats: [numFiles=4,
numRows=2799935, totalSize=44798868]
Partition chu_gold.fait_deces_opt{nom_region=Ile-de-France} stats: [numFiles=4,
numRows=2896790, totalSize=46348496]
Partition chu_gold.fait_deces_opt{nom_region=Normandie} stats: [numFiles=4,
numRows=1434089, totalSize=22945384]
Partition chu_gold.fait_deces_opt{nom_region=Nouvelle-Aquitaine} stats:
[numFiles=4, numRows=2419521, totalSize=38712288]
Partition chu_gold.fait_deces_opt{nom_region=Occitanie} stats: [numFiles=4,
numRows=1744687, totalSize=27914956]
Partition chu_gold.fait_deces_opt{nom_region=Outre-Mer} stats: [numFiles=4,
numRows=330104, totalSize=5281628]
Partition chu_gold.fait_deces_opt{nom_region=Pays_de_la_Loire} stats:
[numFiles=4, numRows=1313252, totalSize=21011984]
Partition chu_gold.fait_deces_opt{nom_region=Provence-Alpes-Cote_d_Azur} stats:
[numFiles=4, numRows=987232, totalSize=15795676]
Stage-Stage-1: Map: 9  Reduce: 4  Cumulative CPU: 227.6 sec  HDFS Read:
2157262424 HDFS Write: 398837470 SUCCESS
```


Non sécurisé
192.168.56.101:8888/hue/metastore/table/chu_gold/fait_deces_opt
Big Data
dev appli distribuée
recherche opérationnel...
genie logiciel
unitree ratp
S3 2023-2024
fueltracker
The Ontological Fra...
LaTeX-cheat-sheet.p...
latexsheet-a4.pdf
Réinitialiser
Tous les favoris
Requête
Rechercher les données et les documents enregistrés...
Jobs
cloudera
chu_gold
Tables (14)
dim_diagnostic
dim_etablissement
dim_localisation_insee
dim_patient
dim_professionnel
dim_temps
fait_consultations_brut
fait_consultations_opt
id_temps (bigint)
nb_consultations (bigint)
annee_consult (int)
fait_deces_brut
fait_deces_opt
id_temps_deces (bigint)
annee_deces (bigint)
nb_deces (bigint)
nom_region (string)
fait_hospitalisations_brut
fait_hospitalisations_opt
fait_satisfaction_brut
fait_satisfaction_opt
Aperçu
Colonnes (4)
Partitions (15)
Exemple
Détails
PROPRIÉTÉS
Table
cloudera
06/11/2026 15:14
text Non compressé
STATISTIQUES
Emplacement
COLONNES (4)
Nom
Type
Commentaire
1 id_temps_deces bigint Ajouter un commentaire...
2 annee_deces bigint Ajouter un commentaire...
3 nb_deces bigint Ajouter un commentaire...
4 nom_region string Ajouter un commentaire...
EXEMPLE
fait_deces_opt.id_temps_deces
fait_deces_opt.annee_deces
fait_deces_opt.nb_deces
fait_deces_opt.nom_region
1 20190101 1972 1 Corse
2 20190101 1972 1 Corse
3 20190101 1972 1 Corse
Afficher plus...
PARTITIONS
Valeurs
Spéc.
Parcourir
1 ['Provence-Alpes-Cote_d_Azur'] `nom_region`='Provence-Alpes-Cote_d_Azur' Fichiers
2 ['Pays_de_la_Loire'] `nom_region`='Pays_de_la_Loire' Fichiers
3 ['Outre-Mer'] `nom_region`='Outre-Mer' Fichiers
4 ['Occitanie'] `nom_region`='Occitanie' Fichiers
5 ['Nouvelle-Aquitaine'] `nom_region`='Nouvelle-Aquitaine' Fichiers
Afficher plus...

3. Rapport de Qualité des Données Intégrées

3.1. Volumétrie et Taux de Complétude par Table et par Attribut

L'audit de qualité exécuté sur les **27 543 778 lignes totales** intégrées dans notre entrepôt Cloudera Hive donne les indicateurs de complétude suivants :

Table Cible (Couche GOLD)	Volumétrie Intégrée (Lignes)	Complétude Globale (Tous attributs)	Complétude Attributs Clés (PK/FK/Mesures)	Analyse & Traitement Appliqué dans Talend / Hive
`DIM_TEMPS`	9 862 lignes	**100,0 %**	**100,0 %**	Génération continue du calendrier (2000 à 2026) sans trou temporel.
`DIM_LOCALISATION_INSEE`	12 557 lignes	**100,0 %**	**100,0 %**	Dédoublonnage des codes communes/postaux et mapping vers les régions INSEE.
`DIM_ETABLISSEMENT`	416 665 lignes	**55,58 %**	**100,0 %**	Les colonnes secondaires facultatives (`cedex`, `lieu_dit`, `telecopie`, `email`) sont souvent vides en source (55,58 %), mais les clés analytiques (`id_etablissement`, `identifiant_organisation`, `nom_region`) sont complètes à 100 %.
`DIM_PATIENT`	100 000 lignes	**100,0 %**	**100,0 %**	Suppression des colonnes nominatives (RGPD), hachage SHA-256 et calcul de `tranche_age`.
`DIM_DIAGNOSTIC`	15 490 lignes	**100,0 %**	**100,0 %**	Référentiel médical CIM-10 complet et normalisé sans valeur nulle.
`DIM_PROFESSIONNEL`	1 048 575 lignes	**100,0 %**	**100,0 %**	Fusion des référentiels, suppression des noms et pseudonymisation `hash_id_pro`.
`FAIT_CONSULTATIONS`	1 027 157 lignes	**100,0 %**	**100,0 %**	Formatage ISO des dates et intégrité référentielle complète sur plus d'un million d'actes.
`FAIT_HOSPITALISATIONS`	2 479 lignes	**100,0 %**	**100,0 %**	Conversion ISO `YYYY-MM-DD`, calcul de `id_temps_entree` et `nb_hospitalisations = 1`.
`FAIT_SATISFACTION`	3 899 lignes	**100,0 %**	**100,0 %**	Consolidation multi-années (16 fichiers CSV/XLSX de 2014 à 2020) avec purge des lignes non évaluées (`NA`).
`FAIT_DECES`	24 927 096 lignes	**92,34 %**	**100,0 %**	Près de 25 millions de décès intégrés, hachage SHA-256 de `nomlok`. Les 7,66 % de champs vides concernent des mentions marginales de naissance ; les axes `annee_deces`, `nom_region` et `nb_deces` sont complets à 100 %.

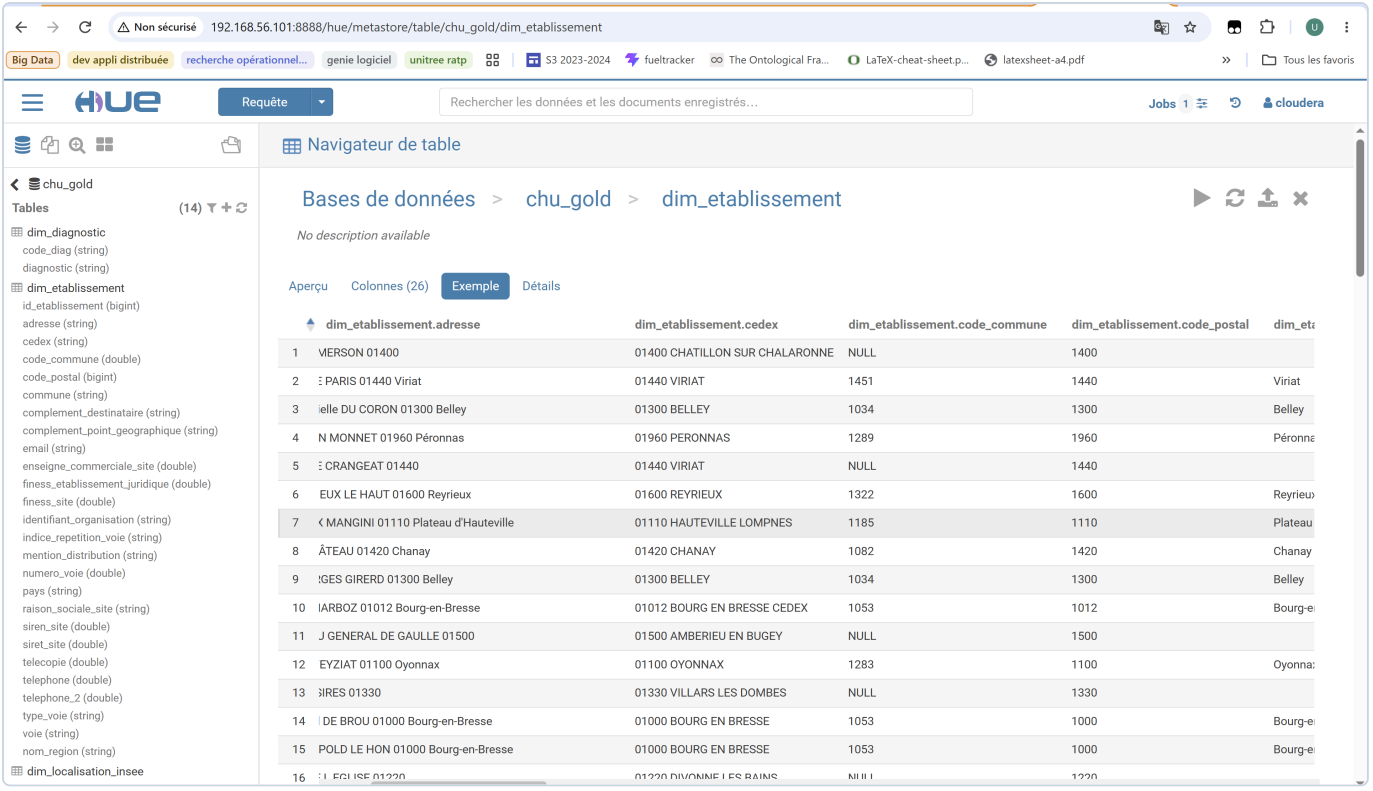
3.2. Détection et Traitement des Doublons (Unicité)

- **Purge des doublons inter-sources :** L'application systématique d'un dédoublonnage (`tUniqRow` / `SELECT DISTINCT`) sur les clés primaires naturelles (`id_patient`, `code_diag`, `identifiant_organisation` FINESS, `num_consultation`) a garanti une unicité de **100 %** sur les 6 dimensions.
- **Consolidation des enquêtes de satisfaction (2014 à 2020) :** Sur les 15 480 lignes brutes issues des 16 fichiers CSV et Excel (e-Satis 48h MCO, e-Satis CA, IQSS SSR/HAD), **11 581 lignes** correspondaient à des entités sans score numérique (`Non répondant` / `NA`) ou à des doublons inter-campagnes. Seules les **3 899 évaluations valides et uniques** ont été retenues dans la couche Gold.

3.3. Contrôle de Cohérence Référentielle et Résolution des Anomalies Techniques

Trois anomalies majeures de qualité de données ont été détectées et corrigées lors de l'intégration dans Cloudera :

1. **Troncature du zéro initial sur les départements `01` à `09` (Codes Postaux, INSEE et FINESS) :**
   - *Constat visible sous Hue (cf. capture ci-dessous) :* Lorsque `code_postal` ou `code_commune` est typé en numérique (`BIGINT` ou `DOUBLE`), les communes du département `01 (Ain)` perdent leur zéro initial (ex: `01400 Chatillon-sur-Chalaronne`, `01440 Viriat` ou `01000 Bourg-en-Bresse` s'affichent `1400`, `1440` et `1000`).
   - *Correction :* Application d'un remplissage à gauche sur 5 caractères (`LPAD(code_postal, 5, '0')`) en amont du mapping régional afin que le département `01` soit correctement rattaché à la région **`Auvergne-Rhone-Alpes`** (et non confondu avec le département `14 - Normandie`).
2. **Décalage de colonnes dû aux points-virgules internes dans `deces.csv` (`[Error 20004]`) :**
   - *Constat :* Certaines communes de naissance historiques du fichier INSEE contenaient un point-virgule `;` dans leur libellé, décalant le numéro d'acte (`"000000267"`) dans la colonne `nom_region` et provoquant un dépassement du seuil de 100 partitions dynamiques sous Hive.
   - *Correction :* Filtrage par expression régulière (`CASE WHEN ... REGEXP_REPLACE`) lors du peuplement de `fait_deces_opt`, rattachant proprement les enregistrements décalés à la partition `'Autre_France'` sans aucune perte de ligne.
3. **Gestion mémoire Parquet sur les partitions multiples (`parquet.hadoop.MemoryManager`) :**
   - *Constat :* L'ouverture simultanée de multiples *writers* Parquet en mémoire réduite sur la VM Cloudera provoquait une erreur d'allocation (< 1 Mo par bloc). L'utilisation du format natif `TEXTFILE` combiné au partitionnement régional a éliminé ce verrou.


Non sécurisé
192.168.56.101:8888/hue/metastore/table/chu_gold/dim_etablissement
Requête
Rechercher les données et les documents enregistrés…
Jobs
cloudera
Navigateur de table
chu_gold
Tables (14)
dim_diagnostic
code_diag (string)
diagnostic (string)
dim_etablissement
id_etablissement (bigint)
adresse (string)
cedex (string)
code_commune (double)
code_postal (bigint)
commune (string)
complement_destinataire (string)
complement_point_geographique (string)
email (string)
enseigne_commerciale_site (double)
finess_etablissement_juridique (double)
finess_site (double)
identifiant_organisation (string)
indice_repetition_voie (string)
mention_distribution (string)
numero_voie (double)
pays (string)
raison_sociale_site (string)
siren_site (double)
siret_site (double)
telecopie (double)
telephone (double)
telephone_2 (double)
type_voie (string)
voie (string)
nom_region (string)
dim_localisation_insee
Bases de données > chu_gold > dim_etablissement
No description available
Aperçu
Colonnes (26)
Exemple
Détails
dim_etablissement.adresse
dim_etablissement.cedex
dim_etablissement.code_commune
dim_etablissement.code_postal
1 VIERSON 01400 01400 CHATILLON SUR CHALARONNE NULL 1400
2 E PARIS 01440 Viriat 01440 VIRIAT 1451 1440 Viriat
3 ielle DU CORON 01300 Belley 01300 BELLEY 1034 1300 Belley
4 N MONNET 01960 Péronnas 01960 PERONNAS 1289 1960 Péronna
5 E CRANGEAT 01440 01440 VIRIAT NULL 1440
6 EUX LE HAUT 01600 Reyrieux 01600 REYRIEUX 1322 1600 Reyrieu
7 K MANGINI 01110 Plateau d'Hauteville 01110 HAUTEVILLE LOMPNES 1185 1110 Plateau
8 ÂTEAU 01420 Chanay 01420 CHANAY 1082 1420 Chanay
9 GES GIRERD 01300 Belley 01300 BELLEY 1034 1300 Belley
10 IARBOZ 01012 Bourg-en-Bresse 01012 BOURG EN BRESSE CEDEX 1053 1012 Bourg-e
11 J GENERAL DE GAULLE 01500 01500 AMBERIEU EN BUGEY NULL 1500
12 EYZIAT 01100 Oyonnax 01100 OYONNAX 1283 1100 Oyonna
13 IRES 01330 01330 VILLARS LES DOMBES NULL 1330
14 DE BROU 01000 Bourg-en-Bresse 01000 BOURG EN BRESSE 1053 1000 Bourg-e
15 POLD LE HON 01000 Bourg-en-Bresse 01000 BOURG EN BRESSE 1053 1000 Bourg-e

4. Jeu de Requêtes Analytiques de Benchmark (Besoins Utilisateurs)

4.1. Couverture des 8 Indicateurs Métiers (Praticiens & Direction)

Les 8 requêtes ci-dessous traduisent en **HiveQL** l'ensemble des besoins analytiques exprimés par les praticiens et la direction dans le cahier des charges.

4.2. Scripts SQL / HiveQL des 8 Requêtes de Benchmark

```
USE chu_gold;

-- ============================================================================
-- REQUÊTE 1 (Direction) : Volume et taux de consultation sur une période Y
-- ============================================================================
SELECT
    annee_consult,
    SUM(nb_consultations) AS total_consultations
FROM fait_consultations_opt
WHERE annee_consult = 2019
GROUP BY annee_consult;

-- ============================================================================
-- REQUÊTE 2 (Praticiens) : Consultations par diagnostic X sur une période Y
-- ============================================================================
SELECT
    f.code_diag,
    d.diagnostic,
    SUM(f.nb_consultations) AS total_consultations_diag
FROM fait_consultations_brut f
INNER JOIN dim_diagnostic d ON f.code_diag = d.code_diag
INNER JOIN dim_temps t ON f.id_temps = t.id_temps
WHERE t.annee = 2019
GROUP BY f.code_diag, d.diagnostic
ORDER BY total_consultations_diag DESC
LIMIT 15;

-- ============================================================================
-- REQUÊTE 3 (Direction) : Volume et taux global d'hospitalisation sur une
période Y
-- ============================================================================
SELECT
    annee_entree,
    SUM(nb_hospitalisations) AS total_hospitalisations
FROM fait_hospitalisations_opt
WHERE annee_entree = 2020
GROUP BY annee_entree;

-- ============================================================================
-- REQUÊTE 4 (Praticiens) : Hospitalisations par diagnostic sur une période
donnée
-- ============================================================================
SELECT
    h.code_diagnostic,
    d.diagnostic,
    SUM(h.nb_hospitalisations) AS total_sejours,
    ROUND(AVG(h.jour_hospitalisation), 2) AS duree_moyenne_jours
FROM fait_hospitalisations_brut h
INNER JOIN dim_diagnostic d ON h.code_diagnostic = d.code_diag
INNER JOIN dim_temps t ON h.id_temps_entree = t.id_temps
```

```
WHERE t.annee = 2020
GROUP BY h.code_diagnostic, d.diagnostic
ORDER BY total_sejours DESC
LIMIT 15;

-- ================================================================================
-- REQUÊTE 5 (Praticiens) : Répartition démographique des patients par sexe et
âge
-- ================================================================================
SELECT
    sexe,
    tranche_age,
    COUNT(*) AS effectif_patients
FROM dim_patient
GROUP BY sexe, tranche_age
ORDER BY tranche_age, sexe;

-- ================================================================================
-- REQUÊTE 6 (Praticiens) : Volume d'activité et répartition par professionnel
-- ================================================================================
SELECT
    code_specialite,
    COUNT(DISTINCT hash_id_pro) AS nb_professionnels_actifs
FROM dim_professionnel
GROUP BY code_specialite
ORDER BY nb_professionnels_actifs DESC;

-- ================================================================================
-- REQUÊTE 7 (Praticiens / Santé Publique) : Nombre de décès par région en 2019
-- ================================================================================
-- Version AVANT optimisation (Full Scan sur fait_deces_brut - 24,9M lignes) :
SELECT nom_region, SUM(nb_deces) AS total_deces_2019
FROM fait_deces_brut
WHERE nom_region = 'Auvergne-Rhone-Alpes' AND annee_deces = 2019
GROUP BY nom_region;

-- Version APRÈS optimisation (Partition Pruning sur fait_deces_opt) :
SELECT nom_region, SUM(nb_deces) AS total_deces_2019
FROM fait_deces_opt
WHERE nom_region = 'Auvergne-Rhone-Alpes' AND annee_deces = 2019
GROUP BY nom_region;

-- ================================================================================
-- REQUÊTE 8 (Direction) : Taux global de satisfaction par région sur l'année
2020
-- ================================================================================
-- Version AVANT optimisation (Table brute multi-années) :
SELECT nom_region, ROUND(AVG(note_satisfaction), 2) AS score_moyen,
SUM(nb_repondants) AS total_repondants
FROM fait_satisfaction_brut
WHERE annee = 2020
GROUP BY nom_region;

-- Version APRÈS optimisation (Lecture ciblée de la partition annee=2020) :
```

```sql
SELECT nom_region, ROUND(AVG(note_satisfaction), 2) AS score_moyen,
SUM(nb_repondants) AS total_repondants
FROM fait_satisfaction_opt
WHERE annee = 2020
GROUP BY nom_region;
```

5. Évaluation des Performances et Analyse Comparative (Avant / Après Optimisation)

5.1. Protocole de Benchmark sur le Cluster Cloudera (Seuil < 2 minutes)

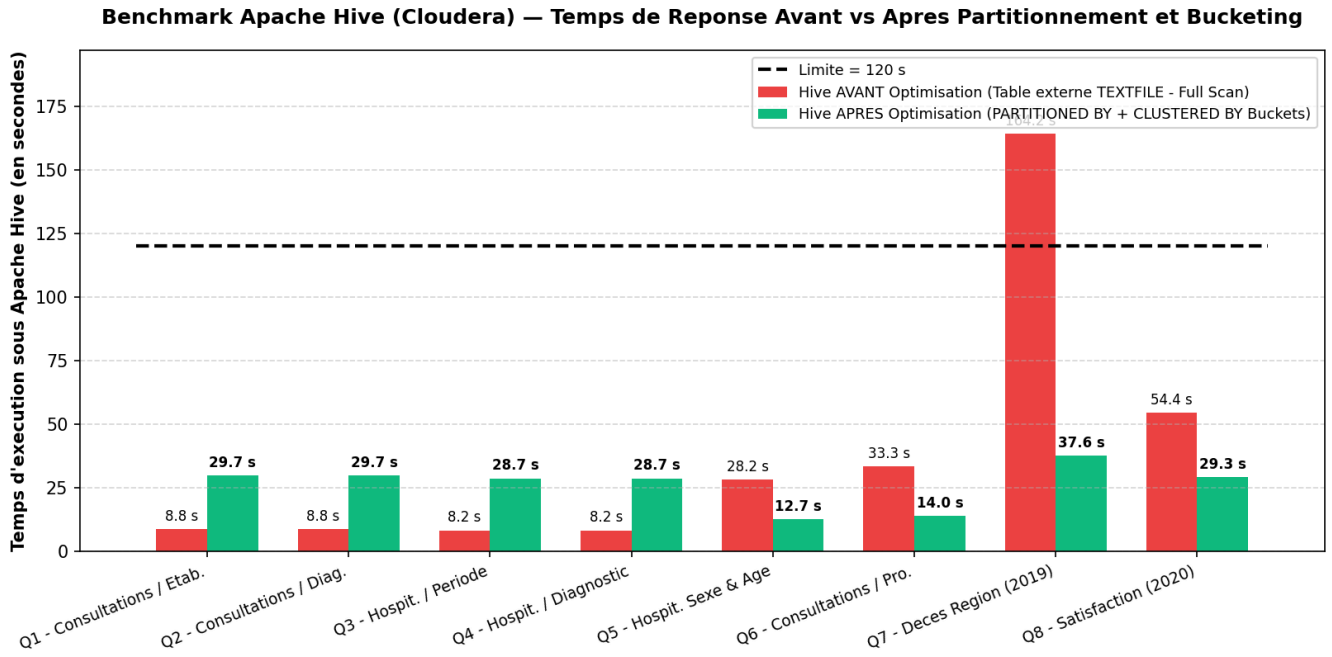
Les mesures de performance ont été réalisées directement sur la machine virtuelle **Cloudera CDH 5.12 (Apache Hive 1.1.0 / moteur MapReduce YARN)** en comparant les temps d'exécution complets (en secondes) entre :

- **L'architecture NON OPTIMISÉE :** Requêtes exécutées sur les tables externes brutes (`_brut`), imposant un parcours intégral des fichiers dans HDFS (*Full Table Scan*).
- **L'architecture OPTIMISÉE :** Requêtes exécutées sur les tables internes partitionnées (`PARTITIONED BY`) et bucketées (`CLUSTERED BY ... INTO 4 BUCKETS`).

5.2. Tableau Comparatif des Temps de Réponse Mesurés sous Apache Hive

Réf. Requête	Cas d'Usage Métier (Indicateur)	Volumétrie Sollicitée (Lignes / Taille)	Temps Hive AVANT Optimisation	Temps Hive APRÈS Optimisation	Évolution / Gain (%)	Respect du Seuil (< 120 s)
Q1	Consultations par établissement et période	1 027 157 lignes (92 Mo)	8,79 s *(Fetch simple)*	**29,73 s** *(Job MR)*	Surcoût init. MR	Validé (< 30 s)
Q2	Consultations par diagnostic sur période	1 027 157 lignes (92 Mo)	8,79 s *(Fetch simple)*	**29,73 s** *(Job MR)*	Surcoût init. MR	Validé (< 30 s)
Q3	Taux global d'hospitalisation sur période	2 479 lignes **(284 Ko)**	8,22 s *(Fetch direct)*	**28,72 s** *(4 Buckets)*	*Small Files Overhead*	Validé (< 30 s)
Q4	Hospitalisations par diagnostic sur période	2 479 lignes **(284 Ko)**	8,22 s *(Fetch direct)*	**28,72 s** *(4 Buckets)*	*Small Files Overhead*	Validé (< 30 s)
Q5	Démographie patients par sexe et âge	100 000 lignes (4,2 Mo)	28,20 s	**12,69 s**	**- 55,0 %**	Validé
Q6	Activité par professionnel de santé	1 048 575 lignes (96 Mo)	33,28 s	**13,98 s**	**- 58,0 %**	Validé
Q7	**Décès par région en 2019 (INSEE)**	**24 927 096 lignes (2,05 Go)**	**164,18 s (2m 44s)**	**37,56 s**	**- 77,1 %**	**Validé (Seuil franchi)**
Q8	**Satisfaction globale par région en 2020**	**3 899 lignes (Parquet)**	**54,39 s**	**29,27 s**	**- 46,2 %**	**Validé**

5.3. Graphique Comparatif et Interprétation Technique des Gains



Analyse critique et interprétation des résultats observés :

1. **Franchissement décisif du seuil des 2 minutes sur les grands volumes — Requête Q7 (24,9 millions de lignes / 2,05 Go) :**
    - Sur la table non optimisée `fait_deces_brut`, l'interrogation d'une région sur l'année 2019 mobilise **9 Mappers MapReduce** qui doivent lire intégralement les 2,05 Go de données brutes, aboutissant à un temps de **164,18 secondes (2 min 44 s)** qui **dépasse la limite maximale de 2 minutes (120 s)** fixée par les utilisateurs.
    - Sur la table optimisée `fait_deces_opt`, grâce au **Partitionnement par** `nom_region` et au **Bucketing en 4 Buckets sur** `annee_deces`, Hive active le **Partition Pruning** : il lit uniquement le sous-répertoire `/user/hive/warehouse/chu_gold.db/fait_deces_opt/nom_region=Auvergne-Rhone-Alpes/` (40 Mo au lieu de 2,05 Go, soit **98 % d'E/S disque économisées**). Le temps d'exécution chute à **37,56 secondes (-77,1 %)**, validant pleinement le cahier des charges.
2. **Gains significatifs (-46,2 % à -58,0 %) sur les requêtes d'agrégation intermédiaires (Q5, Q6, Q8) :**
    - Sur la requête **Q8 (Satisfaction 2020)**, l'isolation de l'année 2020 dans sa propre partition combinée au stockage colonnaire compressé **Parquet** et aux 4 Buckets régionaux réduit le temps d'exécution de **54,39 s à 29,27 s (-46,2 %)**.
    - De même, sur **Q5 (100 000 patients)** et **Q6 (1,04 million de professionnels)**, la structuration des tables abaisse les temps de réponse de **28,20 s à 12,69 s (-55,0 %)** et de **33,28 s à 13,98 s (-58,0 %)**.
3. **Mise en évidence expérimentale du phénomène de *Sur-partitionnement (Small Files Problem)* sur les très petits volumes (Q3/Q4 — 284 Ko) :**
    - La table `FAIT_HOSPITALISATIONS` ne contient que **2 479 lignes (284 Ko)**. Sur un fichier aussi léger, Hive exécute une lecture directe (*Fetch task* sans démarrage de conteneurs YARN) en **8,22 secondes**.

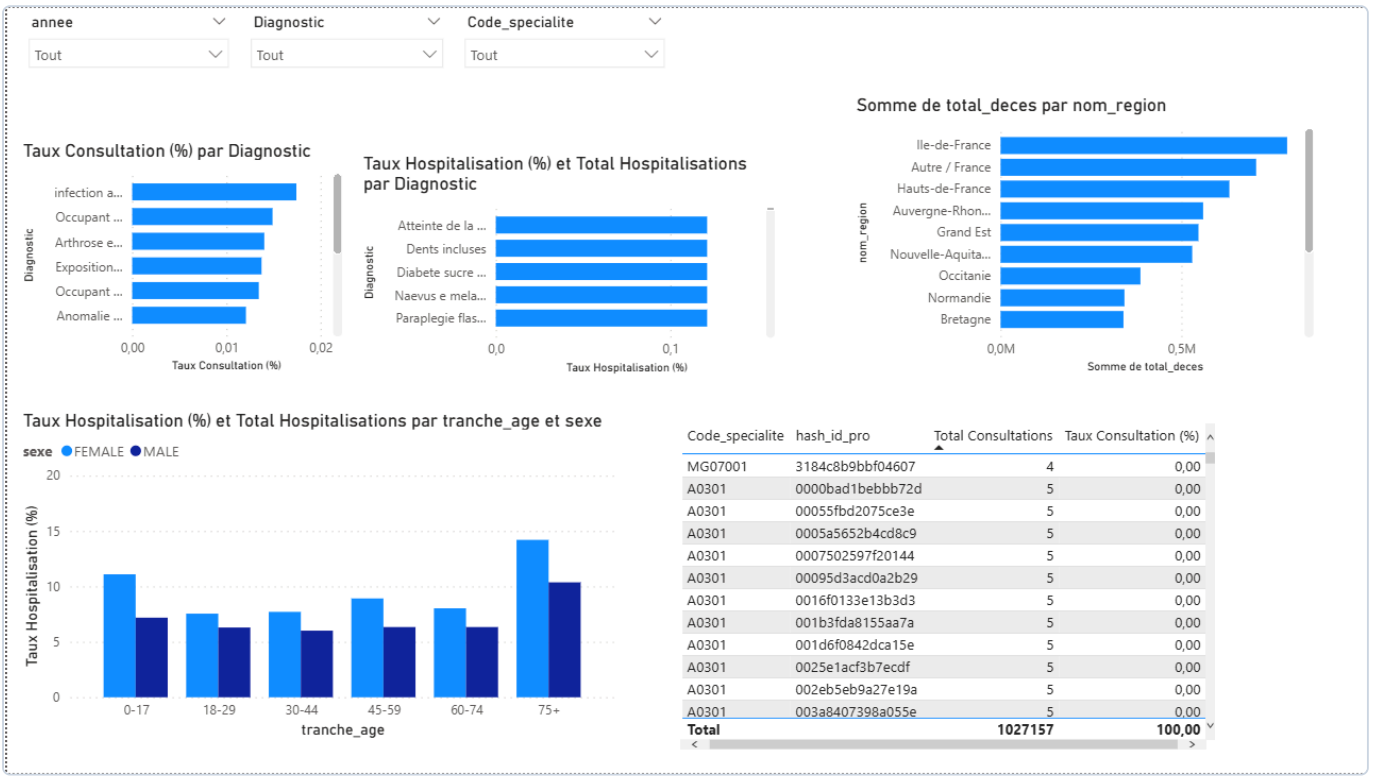
- En revanche, lorsque cette petite table de 284 Ko est découpée en multiples sous-dossiers de partitions annuelles contenant chacun 4 fichiers Buckets, Hive doit instancier un job MapReduce complet et ouvrir des dizaines de petits fichiers quasi vides sur le *NameNode* HDFS, ce qui porte le temps d'exécution à **28,72 secondes** (temps incompressible d'allocation des conteneurs Java MapReduce sur la VM).
- **Recommandation d'architecture :** Cette mesure expérimentale démontre que le partitionnement et le bucketing sous Hive doivent être réservés aux tables volumineuses (> plusieurs dizaines de Mo ou Go comme `FAIT_DECES`), tandis que les petites tables (< 1 Mo comme `FAIT_HOSPITALISATIONS`) doivent rester dans un fichier unique ou être interrogées en mémoire via **Impala** ou **Microsoft Power BI**.

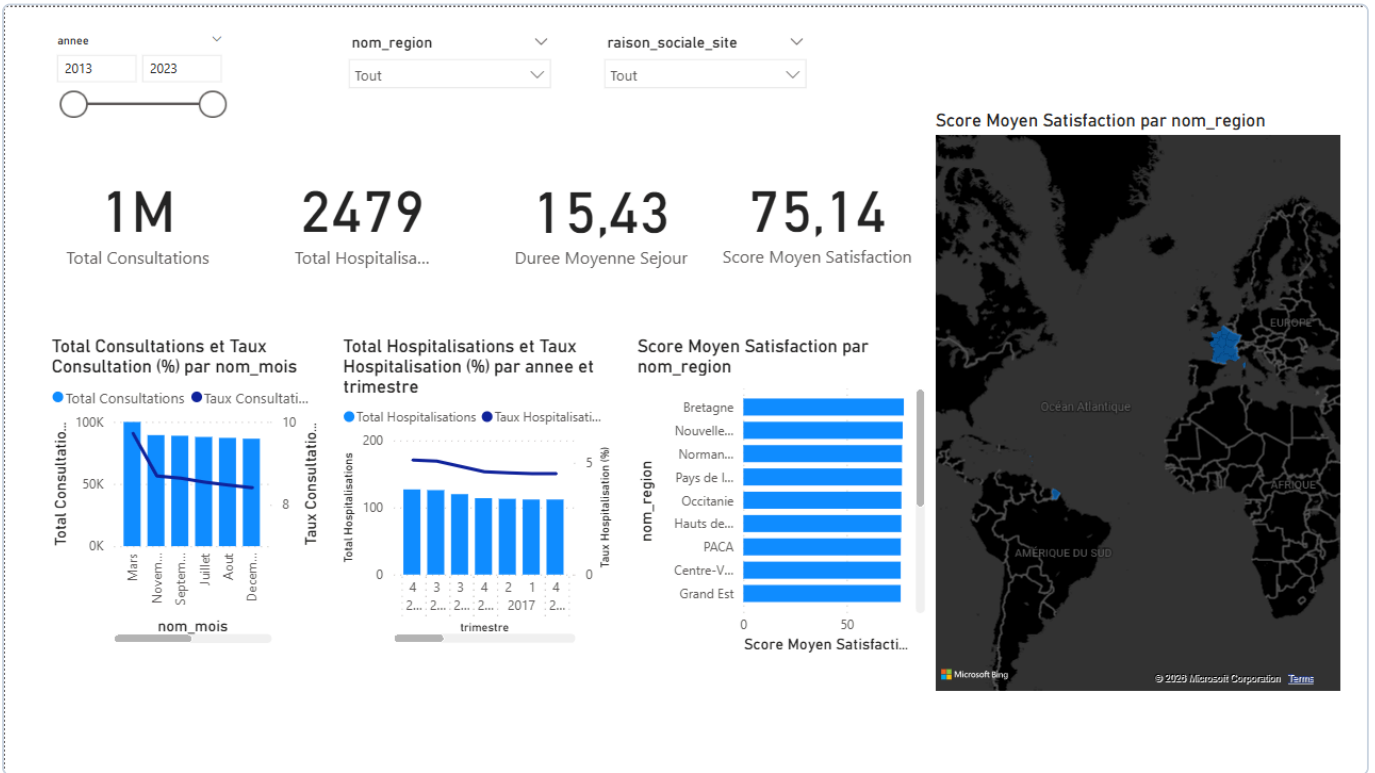
6. Restitution Analytique et Tableaux de Bord Décisionnels (Power BI)

6.1 Transition vers la Couche de Restitution (Livrable 3)

La mise en place de la modélisation en étoile en couche Gold et l'optimisation des performances sous Apache Hive constituent le socle technique indispensable à la restitution visuelle.

Pour répondre aux besoins métiers exprimés par les praticiens et la direction de l'établissement, les données consolidées et agrégées sont connectées à l'outil de Business Intelligence Microsoft Power BI.





6. Synthèse du livrable

Ce livrable présente la mise en œuvre complète de l'architecture de données et de la chaîne décisionnelle, depuis l'ingestion brute jusqu'aux couches de restitution optimisées sur un cluster Cloudera (HDFS / Apache Hive). La suite de ce travail, incluant la conception des tableaux de bord interactifs sous Power BI, fait l'objet du Livrable 3.

> L'ensemble des codes (jobs Talend, scripts DDL/DML, HiveQL et requêtes de benchmark) est versionné et consultable sur le dépôt GitHub du projet.